LangChain Models

1 Introduction & Recap

This chapter revisits LangChain components and zooms into **models**, which are the core engines behind any LLM-powered application. The goal is to understand different types of models and how to use them.

Example: Every chatbot, summarizer, or AI assistant you build depends on a model to generate responses.

---

2 What are Models?

Models are interfaces that process input (text, queries, data) and generate outputs (responses, embeddings, decisions). In LangChain, models are standardized so developers can easily switch providers.
They act as the "brain" of the application.

Example:
Input: "Summarize this article" → Model processes → Output: Short summary

---

3 Plan of Action

The workflow for using models typically includes:

- Selecting the right model
- Setting up API or environment
- Sending structured prompts
- Processing outputs

This structured approach ensures consistency and scalability.

Example: Choose a GPT model → configure API → send prompt → display response in app.

---

4 Language Models

Language models generate human-like text based on prompts. LangChain supports multiple providers like OpenAI and open-source alternatives.
They are mainly used for:

- Chatbots
- Text generation

- Question answering

Example:
Prompt: “Explain photosynthesis in simple terms”
→ Model generates a clear explanation.

---

5 Setup

Setup involves configuring API keys, installing libraries, and initializing the model inside LangChain. This step connects your application to the LLM provider.
Proper setup ensures secure and efficient communication.

Example:
You install LangChain and connect it with OpenAI API using an API key to start sending requests.

---

6 Code Demo (Conceptual Understanding)

This part demonstrates how to call a model using LangChain, pass a prompt, and receive output.
It shows how abstraction simplifies interaction compared to raw API calls.

Example:
Instead of manually formatting API requests, you use a simple LangChain function like:
model.invoke("Tell me a joke")

---

7 Open Source Models

Open-source models provide an alternative to paid APIs. They can run locally or on private servers, offering more control and privacy.
However, they may require more setup and computational resources.

Example:
You use a local LLM instead of OpenAI to build a private chatbot for company data.

---

8 Embedding Models

Embedding models convert text into numerical vectors that capture semantic meaning.
These vectors are used for search, similarity, and retrieval tasks.
They are essential for RAG systems.

Example:
Query: “AI in healthcare”
→ Converted to vector → Compared with stored document vectors → Most relevant documents retrieved.
